INFO 5 – Projet 1 : Analyse mathématique et statistique de textes

Niveau : L1 Maths | Séance 1 sur 4 (3h)

Objectifs de la séance

- Charger et manipuler des chaînes de caractères en Python (algorithmique de base).
- Appliquer des opérations de nettoyage de données via des bijections simples.
- Construire la représentation vectorielle d'un texte et calculer des probabilités empiriques.
- Implémenter des formules mathématiques avancées (Entropie, Indice de coïncidence).

Partie 1 : Lecture et normalisation des données

Avant toute analyse mathématique, il est indispensable de nettoyer les données brutes. La ponctuation, les majuscules et les accents fausseraient nos statistiques.

Exercice 1 : Lire un fichier texte

Créez un fichier texte nommé `test.txt` avec quelques phrases de votre choix. Complétez la fonction ci-dessous pour ouvrir ce fichier et récupérer son contenu.

```python
def lire_fichier(nom_fichier):
    """
    Ouvre un fichier texte en mode lecture avec l'encodement UTF-8.
    Retourne le contenu du fichier sous forme de chaine de caracteres.
    """
    with open(nom_fichier, '___', encoding='utf-8') as f:
        contenu = f.________()
    return contenu
```

Exercice 2 : Normalisation du texte

Nous allons transformer notre texte brut en une suite continue de lettres minuscules non accentuées via une méthode de remplacement manuel des caractères.

```python
def normaliser(texte):
    """
    Convertit en minuscules, supprime les accents manuellement
    et ne conserve que les lettres a-z.
    """
    # 1. Conversion en minuscules


    # 2. Remplacement manuel des accents
    accents = "àâäéèêëîïôöùûüç"
    lettres = "aaaeeeeiiooouuuc"

    # On parcourt les indices pour faire correspondre accent et lettre


    # 3. Filtrage : on ne garde que les caracteres de l'alphabet


    return texte_propre
```

Partie 2 : Représentation vectorielle et Fréquences

Nous allons modéliser notre texte sous la forme d'un vecteur $F = (f_0, f_1, \ldots, f_{25})$ où f_i représente la fréquence de la lettre d'indice i ('a' $\to$ 0, 'b' $\to$ 1, ...).

Exercice 3 : Comptage absolu des lettres

Pour indexer notre tableau, nous utiliserons la fonction `ord(char)` qui renvoie le code entier d'un caractère. L'indice de 'a' devient 0 via : `ord(lettre) - ord('a')`.

```python
def compter_lettres(texte_propre):
    """Renvoie une liste de 26 entiers correspondant aux comptages."""
    counts = [0] * ____

    return counts
```

Exercice 4 : Passage aux fréquences

La fréquence f_i est donnée par $f_i = \frac{\text{counts}[i]}{N}$, où N est le nombre total de lettres. Mathématiquement, la somme des fréquences d'un système complet d'événements vaut 1 : $\sum_{i=0}^{25} f_i = 1$.

```python
def calculer_frequences(counts):
    """Transforme le vecteur des comptages en vecteur de frequences."""

    frequences = []

    # Verification mathematique (aux erreurs d'arrondi pres)
    assert abs(sum(frequences) - 1.0) < 1e-6, "Erreur : La somme n'est pas 1 !"

    return frequences
```

Partie 3 : Analyses statistiques et visualisation (Pour aller plus loin)

Exercice 5 : Histogramme textuel

Pour visualiser la distribution, codez un affichage sous forme de graphique à barres dans la console (1% = 1 étoile $*$).

```python
def afficher_histogramme(frequences):
    """Affiche un histogramme horizontal des frequences."""

        # 1. Retrouver la lettre correspondante a l'indice i

        # 2. Calculer le nombre d'etoiles (mise a l'echelle x100)

        # 3. Affichage (astuce : 'A' * 3 donne 'AAA' en Python)
```

Exercice 6 : Entropie de Shannon

L'entropie H mesure le niveau de "désordre" d'un texte. Plus les lettres sont équiréparties, plus H est grand. La formule est (avec $\log_2$ le logarithme en base 2) :

$$H = -\sum_{i=0}^{25} f_i \log_2(f_i)$$

Attention : le logarithme n'est défini que pour $f_i > 0$.

```python
import math

def calculer_entropie(frequences):
    entropie = 0.0

    return entropie
```

Question : Quelle serait l'entropie d'un texte composé uniquement de la lettre 'a' ?

Exercice 7 : Indice de Coïncidence (Cryptographie)

L'indice de coïncidence calcule la probabilité que deux lettres tirées au hasard dans le texte soient identiques, à partir des comptages **absolus** n_i :

$$IC = \sum_{i=0}^{25} \frac{n_i(n_i - 1)}{N(N-1)}$$

```python
def indice_coincidence(counts):


    ic = 0.0




    return ic
```